

DP-DGAD: A Generalist Dynamic Graph Anomaly Detector with Dynamic Prototypes

Jialun Zheng

Department of Computing
The Hong Kong Polytechnic
University
Hong Kong, HKSAR, China
22069255r@connect.polyu.hk

Jie Liu

Department of Computer Science
City University of Hong Kong
Hong Kong, HKSAR, China
jliu288@cityu.edu.hk

Jiannong Cao

Department of Computing
The Hong Kong Polytechnic
University
Hong Kong, HKSAR, China
csjcao@comp.polyu.edu.hk

Xiao Wang

School of Software
Beihang University
Beijing, China
xiao_wang@buaa.edu.cn

Hanchen Yang*

Department of Computing
The Hong Kong Polytechnic
University
Hong Kong, HKSAR, China
hanchen.yang@connect.polyu.hk

Yankai Chen

School of Computer Science
McGill University
Montreal, QC, Canada
yankaichen@acm.org

Abstract

Dynamic graph anomaly detection (DGAD) is essential for identifying anomalies in evolving graphs across domains such as finance and social networks. Recently, generalist graph anomaly detection (GAD) models have shown promising results. They are pretrained on multiple source datasets and generalize across domains. While effective on static graphs, they struggle to capture evolving anomalies in dynamic graphs. Moreover, the continuous emergence of new domains and the lack of labeled data further challenge generalist DGAD. Effective cross-domain DGAD requires both domain-specific and domain-agnostic anomalous patterns. Importantly, these patterns evolve temporally within and across domains. Building on these insights, we propose a DGAD model with Dynamic Prototypes (DP) to capture evolving domain-specific and domain-agnostic patterns. Firstly, DP-DGAD extracts dynamic prototypes, i.e., evolving representations of normal and anomalous patterns, from temporal ego-graphs and stores them in a memory buffer. The buffer is selectively updated to retain general, domain-agnostic patterns while incorporating new domain-specific ones. Then, an anomaly scorer compares incoming data with dynamic prototypes to flag both general and domain-specific anomalies. Finally, DP-DGAD employs confidence detection guided memory buffer updating for effective adaptation to target domain. Extensive experiments demonstrate state-of-the-art performance across ten real-world datasets from different domains.

CCS Concepts

• Information systems → Data mining; • Computing methodologies → Machine learning.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792268>

Keywords

Anomaly Detection; Dynamic Graph Anomaly Detection; Graph Neural Networks.

ACM Reference Format:

Jialun Zheng, Jie Liu, Jiannong Cao, Xiao Wang, Hanchen Yang, and Yankai Chen. 2026. DP-DGAD: A Generalist Dynamic Graph Anomaly Detector with Dynamic Prototypes. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792268>

1 Introduction

Graph anomaly detection (GAD) identifies unusual nodes or edges based on their feature distributions. It has wide applications in online web platforms, such as fraud detection, financial transaction analysis, and social networks [8, 26, 35]. However, real-world graphs are often evolving [11, 17, 55, 56], such as new accounts being added or existing profiles being updated in the social networks. Unlike static GAD, dynamic graph anomaly detection (DGAD) focuses on capturing not only anomalous patterns but also their temporal evolution. Therefore, DGAD is a challenging yet representative of real-world scenarios compared with that of GAD [12, 21, 42, 46].

With the advancement of deep learning, several methods have been developed to address DGAD problems [35]. Early work, such as TADDY [20], employs transformers to capture both structural and temporal patterns for anomaly detection. More recently, approaches like GeneralDyG [45] and FALCON [3] have introduced unsupervised DGAD models. They assume all training datasets have only normal samples and detect anomalies by identifying samples that deviate from normal distributions. Despite promising results, existing DGAD models follow a “one model per domain” paradigm, limiting generalization to new domains. However, datasets from different domains have distinct anomalous patterns, feature distributions, and structures, leading to domain shift [28, 33, 52]. It hinders the generalization capability of current DGAD models across datasets from different domains. Consequently, there is a growing demand for a unified DGAD model that can adapt to multiple datasets from different domains.

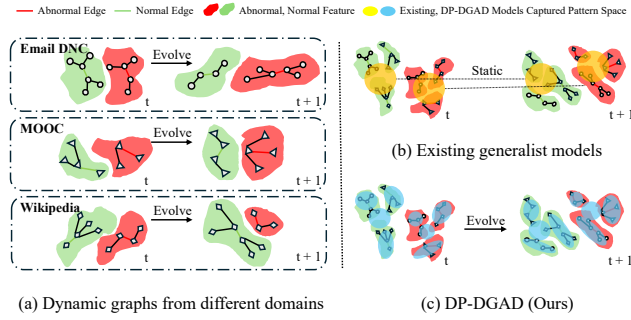


Figure 1: (a) Anomalous patterns change over time. (b) Existing generalist methods are static, capturing only domain-agnostic patterns. (c) Our method, DP-DGAD, captures both domain-agnostic and domain-specific patterns.

Recently, generalist models for static graphs, such as ARC [19] and AnomalyGFM [27], have emerged as promising solutions. These generalist models are pretrained on source datasets and generalize to target datasets from different domains. However, these models cannot be directly applied to DGAD due to the following challenges: **1) Temporally evolving anomalous pattern.** Current generalist detectors for static graphs assume static spatial anomalies, neglecting the temporal dynamics of evolving graphs. In contrast, anomalous patterns in dynamic graphs constantly change, such as node and edge feature distributions and structures (see Fig. 1(a)). These changes significantly complicate generalization. **2) Continuous domain shift.** The continuous emergence of new domains presents a challenge: models must balance domain-specific and general patterns, often sacrificing one for the other (as shown in Fig. 1(b)). This results in unstable performance across datasets. **3) Lack of labeled datasets.** Existing methods struggle to incorporate new patterns from target datasets due to the lack of labeled data, which is labor-intensive. As a result, they rely only on patterns from source datasets, which may differ from those in target datasets and cause performance drops.

To tackle the above challenges, we introduce DP-DGAD, a generalist Dynamic Graph Anomaly Detection method based on Dynamic Prototypes. Here, dynamic prototypes refer to representations that reflect the abnormal and normal classes, which evolve over time. The basic idea is illustrated in Fig. 1(c). DP-DGAD captures both domain-specific and domain-agnostic patterns via dynamic prototypes as they evolve over time. These patterns further help to mitigate negative effects from continuously emerging domains [47, 48].

Specifically, our approach generates dynamic prototypes by first capturing spatial anomalies from ego-graphs. These dynamic prototypes are updated with edge-level temporal attributes, creating representations that capture the evolution of anomalous patterns. Afterward, dynamic prototypes are saved in a memory buffer that balances domain-agnostic prototypes for generalizability with new domain-specific ones for adaptation to new domains, thereby effectively countering continuous domain shifts. From these prototypes, we extract normal and abnormal distributions for anomaly scoring, enabling confident detection [32] for pseudo-labeling on target datasets. Using pseudo-labels, DP-DGAD adapts to new domains

without updating model parameters. Instead, it update only the memory buffer, enabling parameter-free adaptation that is highly practical for label-scarce scenarios. Extensive experiments on ten real-world datasets validate the state-of-the-art performance of DP-DGAD. The key contributions are summarized as follows:

- We, for the first time, formally define and address the generalist DGAD problem. The problem aims to train a DGAD model on multiple labeled source datasets from different domains and generalize to unlabeled target datasets.
- We introduce DP-DGAD, a novel generalist DGAD model based on Dynamic Prototypes. Our approach extracts and updates prototypes that capture evolving spatial and temporal patterns. We selectively retain prototypes relevant to new domains to ensure balanced and adaptive learning. Then, it leverages high-confidence detections from target datasets as pseudo-labels, enabling adaptation to target domains.
- Experiments show our method achieves the best performance among all advanced baselines on ten cross-domain datasets. Our code is now available.

2 Related Work

2.1 Dynamic Graph Anomaly Detection

Traditional Graph Anomaly Detection (GAD) [18, 23] aims to identify nodes or edges with abnormal patterns. Notwithstanding, it struggles with evolving graphs, where node features, edge attributes, and structures change over time. Dynamic Graph Anomaly Detection (DGAD) [21] addresses this limitation and is applied in inherently dynamic fields such as traffic [8] and social networks [35]. To capture these evolving patterns, several deep learning methods have been proposed. For example, TADDY [20] uses transformers to encode spatial and temporal changes. Followed by StrGNN and DynAnom [1, 9] that focus on local subgraphs to efficiently detect structural anomalies. Additionally, STGAN [8] introduces a convolutional adversarial network to handle evolving anomalies, including structural and feature changes. Alternatively, models like SAD and CoLA [20, 31] use semi-supervised learning with partial labels [5]. However, aforementioned approaches cannot be directly applied to real-world scenarios due to a lack of labeled datasets.

Unsupervised DGAD [45], which operates without labels, has gained attention for its practicality. Most unsupervised DGAD methods assume training data contains only normal samples and learn node or edge embeddings to model this normal distribution. Among them, FALCON [3] leverages fine-grained temporal information through enhanced sampling and embedding. Then it improves generalization via an attention alignment optimized by contrastive learning loss [38, 58]. Instead of focusing on embedding, SLADE [15] models evolving long-term interactions to learn normal patterns and find anomalies. However, these methods typically follow a "one model per domain" paradigm and struggle to generalize across different domains [4, 53].

2.2 Generalist Anomaly Detector

First explored in image anomaly detection, the generalist anomaly detector paradigm involves pretraining models on multiple source datasets to generalize across different domains [6, 40, 57]. To generalize learned patterns to new domains, the model must capture

invariant anomalous patterns [2, 7, 36] across domains. However, compared to image data, graph data exhibit more complex and variable structures that also change across domains [10, 43, 49, 51].

Recently, generalist approaches for GAD [19, 24, 27, 39] have emerged as a promising direction due to their cross-domain generalizability. These methods commonly use unified strategies, such as normalization [16, 54], prototype-based approaches [30, 34, 37] or prompt-based [22] method to address domain differences. Although results are promising, their unified strategies often lead to unbalanced performance, excelling on some datasets while performing poorly on others. Moreover, they are not suited for dynamic graphs, which require capturing anomalous patterns in an evolving manner.

3 Preliminaries

Notations. Dynamic graph can be denoted as $\mathcal{G} = \{G_1, G_2, \dots, G_T\}$ with T being number of time intervals, and each interval consists M number of timestamps. Specifically, for each time interval $t \in \{1, 2, \dots, T\}$, we have $G_t \in \{G_1, G_2, \dots, G_T\}$. We further have $G_t = (V_t, E_t)$, V_t is a node set consisting of entities, E_t is an edge set consisting of spatial relations. Additionally, $A^{V_t} \in \mathbb{R}^{N^{V_t} \times N^{V_t}}$ and $A^{E_t} \in \mathbb{R}^{N^{E_t} \times N^{E_t}}$ are the node adjacency matrix and edge adjacency matrix during time interval t , respectively, where $N^{V_t} = |V_t|$, $N^{E_t} = |E_t|$ are the number of nodes and edges. We further have normal edge set E_t^n and abnormal edge set E_t^a , where each edges have label $y \in \{0, 1\}^{N^{E_t}}$ with 0 denoting the normal edges and 1 denoting the abnormal edges. During certain time interval t , nodes and edges will record data as features that can be represented as $X^{V_t} \in \mathbb{R}^{N^{V_t} \times D^V \times M}$, $X^{E_t} \in \mathbb{R}^{N^{E_t} \times D^E \times M}$ where D^V and D^E are the dimension of nodes and edges' data, respectively.

Continuous Domain Shift. Given $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_{S+K}\}$ being a collection of $(S + K)$ dynamic graph from different domains, where $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_S\}$ are from source domain and $\{\mathcal{G}_{S+1}, \mathcal{G}_{S+2}, \dots, \mathcal{G}_{S+K}\}$ are from target domains. Each domain's dynamic graph $\mathcal{G}_i$ can be represented as $\mathcal{G}_i = \{G_1, G_2, \dots, G_{T_i}\}$ with T_i time intervals. For two dynamic graphs from different domains, $\mathcal{G}_i$ and $\mathcal{G}_j$ where $i \neq j$, we define domain shift as the difference in distributions of node and edge features, graph structural, such as the adjacency matrix or the mapping from edges to anomaly labels. This domain shift continuously occurs between any two dynamic graphs in the dataset collection $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_{S+K}\}$.

Problem Definition. This work aimed to develop a generalist anomaly detector to detect anomalous edges at each timestamp within a specified time interval. On top of it, the detector is trained on labeled source datasets and then generalizes to unlabeled target datasets. Specifically, given S number of source datasets, our goal is to learn an anomaly scoring function Ψ such that for a specific target dynamic graph $G_k = (V_k, E_k)$, we can predict the label set y_k of E_k using function Ψ :

$$\Psi(\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_S) = y_k. \quad (1)$$

Notably, the above function Ψ is trained using labels exclusively from the source datasets, while the target datasets remain unlabeled to simulate a real-world scenario.

4 Methodology

DP-DGAD aims to capture both domain-agnostic, and domain-specific evolving anomalous patterns. This allows patterns learned from labeled source datasets to generalize to unlabeled target datasets across different domains. Specifically, as shown in Fig. 2, DP-DGAD first extracts evolving patterns using dynamic prototype extraction and stores them in a memory buffer. Then, during cross-domain anomaly scoring, it retains general patterns shared among pre-trained source datasets while incorporating domain-specific ones. These dynamic prototypes are compared with embeddings to calculate anomaly scores. After that, pretrained anomaly scorer produces high confidence detection on the target domain based on entropy ranking. These confident detections provide pseudo-labels for unlabeled edges and enable parameter-free updating of memory buffer through prototype alignment loss. Finally, updated memory buffer accompanied with generalist model perform anomaly detection on target datasets.

4.1 Dynamic Prototype Extraction

In this section, our goal is to extract dynamic prototypes as the evolving representations of normal and anomalous patterns. Particularly, we align prototypes with both abnormal and normal patterns to fully exploit anomaly discriminability. This approach enables a more effective capture of evolving anomalous patterns over time.

For each edge e_i , its anomalous pattern can be captured by the ego-graph consisting of its neighbors, thereby reflecting how it deviates from or resembles them [27]. To capture not only the anomalous pattern but also its evolution over time, temporal ego-graphs are utilized. These graphs [45] extract k -hop neighboring edges occurring on or before timestamp t around edge e_i and form a subgraph G_{ego} . The temporal ego-graph is then input into a simple backbone model consisting of a Graph Neural Network (GNN), followed by a transformer to retrieve representations. The GNN first outputs representation $H_l \in \mathbb{R}^{|E_{ego}| \times d}$ as follows:

$$H_l = \sigma(A_{ego} H_{l-1} W_1^l + H_{l-1} W_2^l), \quad (2)$$

where d is the dimension of representation and l is the GNN layer index. $W_2^l, W_1^l \in \mathbb{R}^{D^{l-1} \times D^l}$ are learnable parameter matrix of layer l and σ is the activation function. A_{ego} refers to the edge adjacency matrix of the ego-graph. To make sure the representation captures domain agnostic features, a residual module is adapted here to process edge-specific representation $h_i \in H_l$ as follows:

$$h'_i = h_i - \frac{1}{|E_{ego}|} \sum_{e_j \in E_{ego}} h_j. \quad (3)$$

Then, the processed H'_l is fed into a transformer. This transformer's role is to identify the similarity between edge e_i and its k -hop neighbors. In this way, we encode the spatial anomalous patterns of e_i as they evolve over time into its embedding z_i .

$$z_i = \sum_{h'_j \in H'_l} \frac{\exp(\frac{\langle w_Q h'_i, w_K h'_j \rangle}{\sqrt{d_{out}}})}{\sum_{h'_j \in H'_l} \exp(\frac{\langle w_Q h'_i, w_K h'_j \rangle}{\sqrt{d_{out}}})} w_V h'_j, \quad (4)$$

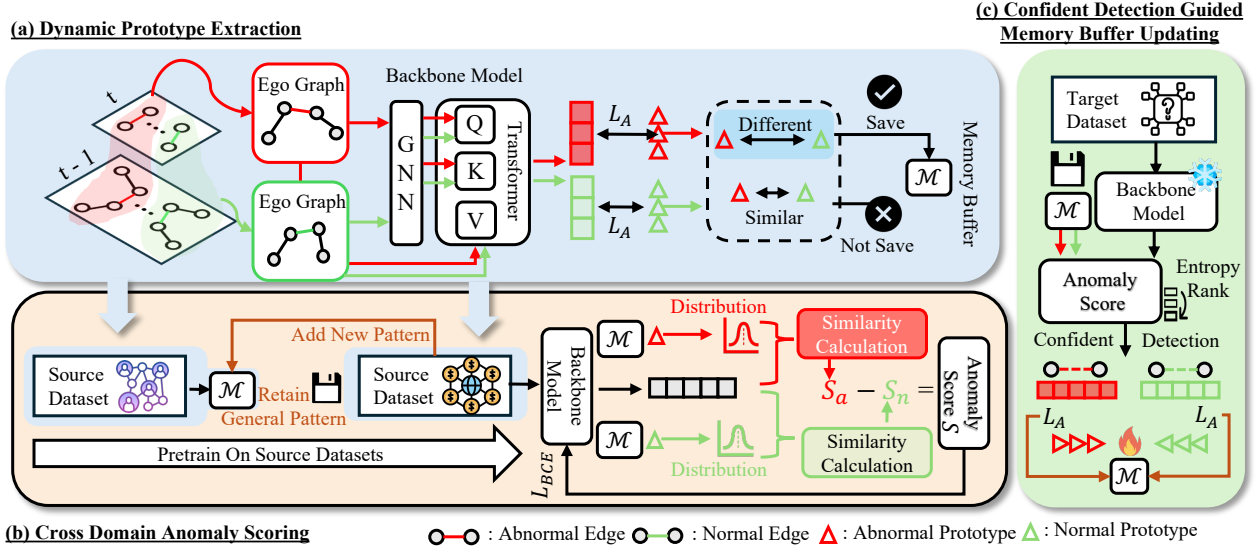


Figure 2: General framework of DP-DGAD. (a) Starting with the first source dataset, we extract ego-graphs to capture temporal patterns and store the most distinct prototype pairs in a memory buffer. (b) As we pretrain on the following source datasets, new domain-specific patterns are added while general domain-agnostic patterns are retained. Prototype distributions are then compared with edge embeddings for anomaly scoring. (c) On target datasets, the pretrained anomaly scorer selects confident low-entropy detections for pseudo-labeling and memory buffer updating, aligning prototypes with the target domain. While the generalist model’s parameters remain frozen.

where $\langle \cdot, \cdot \rangle$ denotes the dot product, d_{out} refers to the dimension of the output representation z_i of edge e_i . w_Q, w_K, w_V refer to the Query, Key, and Value matrix.

After obtaining the representation of each edge, we align dynamic prototypes with both the abnormal and normal edges’ representations. Formally, we have $p_n \in \mathbb{R}^{d_p}$ as the dynamic normal prototypes while $p_a \in \mathbb{R}^{d_p}$ represents the dynamic abnormal prototypes. Both for the abnormal e_i in E_t^a and that in E_t^n , we can obtain the ego-graph representation. We denote the representation of abnormal edges as Z_a and the representations of normal edges as Z_n . To align p_a and p_n with Z_a and Z_n , we define alignment loss function as follows:

$$L_A = \sum_{i=1}^{|E_t|} I_{y_i=0} \|z_i - p_n\|_2^2 + I_{y_i=1} \|z_i - p_a\|_2^2, \quad (5)$$

where $I_{y_i=0}$ represents an indicator function. This function returns a value of 1 when the condition $y_i = 0$ is met, and 0 otherwise.

These aligned dynamic prototypes are further stored in the dynamic prototype buffer $\mathcal{B}$ that has size M , set as 10% of the dataset size. The buffer is updated during each iteration to capture diverse aspects of the abnormal and normal patterns as possible. Due to limitations in size and memory, we retain only the most discriminative prototypes. This is achieved by ranking all prototypes currently in the buffer in ascending order based on the difference score. The prototypes that are found to be the least distinct (i.e., most similar to others) are then replaced. The difference score is calculated as the mean Euclidean distance between pairs of prototypes:

$$s_d = \frac{1}{d_p} \sum_{m=1}^{d_p} \|p_a^m - p_n^m\|_2, \quad (6)$$

where d_p refer to the dimension of prototypes, p_a^m, p_n^m refer to the corresponding dimension m ’s feature in prototypes. In each training epoch, the system identifies the pair of prototypes that exhibits the highest difference score. This most distinct pair is then used to set the initial values for p_n and p_a for that specific epoch. This strategy ensures that the dynamic prototypes are continuously refined to be more distinguishable.

4.2 Cross Domain Anomaly Scoring

Distribution-based anomaly scoring surpasses traditional binary classifiers in its ability to generalize across different domains. This is because it captures higher-order statistics of data distributions by comparing data with both abnormal and normal distributions. However, our current dynamic prototypes are domain-specific, which prevents their direct application across different domains. To generalize them to new domains, we identify and save domain-agnostic ones, while adding new prototypes from new domains. Subsequently, we can build an anomaly scoring module that leverages these enhanced prototypes.

For identifying domain-agnostic patterns, they should already be present in the memory buffer, which stores information from previously encountered source datasets. Additionally, they should also exhibit similarity to the feature representations found in the subsequent source dataset. The similarity s_e is calculated as the

mean Euclidean distance between the dynamic prototype pair and the representations of edges in the new domain dataset:

$$s_e = \frac{1}{|E_t|} \sum_{i=1}^{|E_t|} \|z_i - p_n\|_2 + \|z_i - p_a\|_2. \quad (7)$$

Notably, the representation z_i is projected to the same dimension as that of the dynamic prototypes. A lower distance indicates higher similarity, meaning more general prototypes. This will be combined with the difference score in the following manner:

$$s_r = \lambda_d s_d - \lambda_e s_e, \quad (8)$$

where λ_d and λ_e are parameters used to control the contribution of the two scores. A higher s_r indicates greater pairwise difference and similarity to the new source dataset, representing more general domain-agnostic patterns. For new domain patterns, we rank buffer prototypes by s_r in ascending order and replace the lowest-scoring pair with one from the new domain. We then initialize p_n and p_a using the highest s_r pairs to ensure dynamic prototypes capture both domain-agnostic and domain-specific patterns.

With these dynamic prototypes in the memory buffer, we build an anomaly scoring module by measuring edge representation similarity to both normal and abnormal distributions. Top prototype pairs alone cannot fully represent distributions, so we iteratively update statistical measures (mean and covariance). Each iteration incorporates new data and existing statistics to capture evolving distributions. The mean update is as follows:

$$\mu_{n,t} = \alpha \mu_{n,t-1} + (1 - \alpha) \sum_{i=1}^M p_{n,i}, \quad (9)$$

$$\mu_{a,t} = \alpha \mu_{a,t-1} + (1 - \alpha) \sum_{i=1}^M p_{a,i}, \quad (10)$$

where μ_n and μ_a are the mean values for normal and abnormal distributions. α is a momentum parameter to control the ratio of updating based on prototypes and the existing mean value.

For covariance updates, we first calculate center embedding, which captures how individual prototypes differ from the overall average. Then, we use center embeddings to update the covariance that inherently measures how data points vary together around their mean values:

$$C_n = [p_{n,1} - \mu_{n,t}, p_{n,2} - \mu_{n,t}, \dots, p_{n,M} - \mu_{n,t}]^T, \quad (11)$$

$$C_a = [p_{a,1} - \mu_{a,t}, p_{a,2} - \mu_{a,t}, \dots, p_{a,M} - \mu_{a,t}]^T, \quad (12)$$

$$\Sigma_{n,t} = \Sigma_{n,t-1} + (1 - \alpha) \frac{C_n^T C_n}{\max(1, M - 1)}, \quad (13)$$

$$\Sigma_{a,t} = \alpha \Sigma_{a,t-1} + (1 - \alpha) \frac{C_a^T C_a}{\max(1, M - 1)}, \quad (14)$$

where C_n and C_a are the center embeddings, and, Σ_n and Σ_a are the covariance values.

Then we will calculate the similarity score s_a, s_n to the abnormal distribution and the normal distribution, respectively:

$$s_{n,i} = z_i^T \mu_n - \lambda_n z_i^T \Sigma_n z_i, \quad (15)$$

$$s_{a,i} = z_i^T \mu_a - \lambda_a z_i^T \Sigma_a z_i, \quad (16)$$

where λ_a, λ_n is the learnable parameter related to memory buffer. Next, we compute the anomaly score $s_i = s_{a,i} - s_{n,i}$, which measures the similarity gap to determine if an edge is closer to the abnormal or normal distribution. This score is then converted into a probability (p_i) using a sigmoid function, representing the likelihood of the edge being anomalous. The overall learning objective combines binary cross-entropy loss with the previous alignment loss (L_A):

$$L_{BCE} = -\frac{1}{|E_t|} \sum_{i=1}^{|E_t|} y_i \log(p_i) + (1 - y_i) \log(1 - p_i), \quad (17)$$

$$L = \lambda_{BCE} L_{BCE} + \lambda_A L_A, \quad (18)$$

where λ_A and λ_{BCE} are parameters controlling the contribution ratio of these two losses to the final loss.

4.3 Confident Detection Guided Memory Buffer Updating

For unlabeled target data, we generate high-confidence detection using the pretrained scorer for pseudo-labeling, which provides reliable supervision for memory buffer updating. These confident detections update the memory buffer with domain-specific prototypes, which guide detection on the target domain. Notably, the generalist backbone model parameters remain frozen, with only the prototypes in the memory buffer being updated.

We first input the target data into the pretrained anomaly scoring to obtain the probability p_i . However, not all detections are highly confident or close to the ground truth. Therefore, we use the entropy value [32] to reflect the model's detection confidence, which is calculated as follows:

$$\mathcal{H}(p_i) = -p_i \log(p_i + \epsilon) + (1 - p_i) \log(1 - p_i + \epsilon), \quad (19)$$

where ϵ is a small constant added for numerical stability. Lower entropy indicates higher confidence in the model's classification of the sample as normal or abnormal. The top N_{con} abnormal and normal detection pairs with the lowest entropy are selected as confident detections. After that, we use them as pseudo-labels for their corresponding edges for memory buffer updating.

We inject the domain knowledge from high-confidence pseudo-labels into our dynamic prototypes via the alignment loss L_A . This loss exclusively updates the prototypes in the memory buffer, keeping the generalist backbone frozen. The updated prototypes and the frozen model are then used for final anomaly detection.

5 Experiments

5.1 Experimental Setup

Datasets. We use 10 real-world datasets [15, 20, 44] from various domains for evaluation, and the detailed statistics of datasets are presented in Table 1. Details of datasets are listed in Appendix A.1.

Table 1: Statistics of datasets

Dataset	Domain	#Nodes	#Edges	Avg. Degree
Wikipedia	Online Collaboration	9,227	157,474	34.13
Bitcoin-OTC	Transaction Network	5,881	35,588	12.10
Bitcoin-Alpha	Transaction Network	3,777	24,173	12.80
Email-DNC	Online Communication	1,866	39,264	42.08
UCI Messages	Online Communication	1,899	13,838	14.57
AS-Topology	Autonomous Systems	34,761	171,420	9.86
MOOC	Online Courses	7,074	411,749	116.41
Synthetic-Hijack	Email Spam	986	333,200	38.11
TAX51	Transaction Network	132,524	467,279	7.05
DBLP	Citation Network	25387	119899	9.44

Evaluation Metrics and Baselines. We will utilize AUROC and AUPRC, two widely used metrics that have also been adapted in other DGAD models. For both metrics, a higher value denotes a better performance.

As for the baselines, we compare DP-DGAD with several recent DGAD models, GeneralDyG [45], FALCON [3], SLADE [15], SAD [31], TADDY [20]. Generalists for static graphs such as ARC [19], AnomalyGFM [27], UNPrompt [24], and GraphPrompt [22] are also adapted for comparison. All generalist GAD models are adapted to process dynamic graphs by incorporating the same GNN and transformer backbone as DP-DGAD. This modification was made to enable a fair comparison, with all other modules unchanged.

Implementation Details. All methods undergo pretraining on two source datasets, Wikipedia and MOOC [20], chosen because they have available ground truth anomalies. Subsequently, they are fine-tuned on eight target datasets, which contain only normal edges and no abnormal ones. Following this, testing is performed on the same eight target datasets, but with injected anomalies¹.

Parameters for ego-graph extraction, GNN, and the transformer are aligned with those of GeneralDyG [45]. The memory buffer size $\mathcal{M}$ is configured as 10% of the data size. The number of confident detections for pseudo-labeling, N_{con} , is 10% of the data size. The momentum α is set to 0.9. The value of p is varied at 1%, 5%, and 10%. The loss ratios λ_A and λ_{BCE} are set to 0.1 and 0.9, respectively, while λ_d and λ_e are set to 0.3 and 0.7.

5.2 Experimental results

We compared our method, DP-DGAD, with existing Dynamic Graph Anomaly Detection (DGAD) and generalist Graph Anomaly Detection (GAD) methods across eight target datasets from diverse domains. The comparison results, presented in Table 2, lead to several key observations: **(1) DP-DGAD consistently achieves the highest AUROC and AUPRC among all baselines across the eight datasets.** For instance, on Email-DNC, DP-DGAD reached an AUPRC of 34.18, significantly outperforming the second best,

AnomalyGFM, at 22.84. These improvements are substantial, particularly in complex datasets such as TAX51 and DBLP and under low anomaly ratios scenarios. DP-DGAD’s state-of-the-art performance, especially at low anomaly ratios, is attributed to its memory buffer retaining domain-agnostic samples. This buffer enables our model to review anomalous patterns, thus mitigating insufficient anomaly data. **(2) DP-DGAD achieves stable performance under continuous domain shifts across datasets.** As shown in Fig. 3, DP-DGAD exhibits the lowest coefficient of variation, indicating superior stability. Generalist models, while adapting to capture domain-agnostic patterns, don’t match DP-DGAD’s performance due to neglecting domain-specific and evolving patterns. However, they are generally better than DGAD models that only capture domain-specific, but struggle to capture domain-agnostic patterns. **(3) DP-DGAD’s performance increases with the anomaly ratio.** This trend, also seen in other baselines, occurs because a higher anomaly ratio enables models to capture more diverse anomaly patterns, leading to improved results. Consequently, subsequent experiments will use 10% anomaly ratio datasets for visualization.

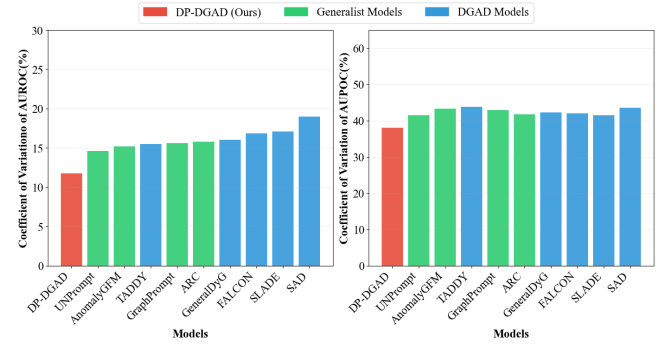


Figure 3: Model performance under continuous domain shift.

5.3 Ablation Study

Effectiveness of Important Module. In this section, we conduct experiments by removing key components in DP-DGAD to study their effectiveness: (1) **w/o DPA** replaces the dynamic prototype-based anomaly scoring with a normal binary classifier. (2) **w/o DPAD** sets λ_e to 0 and λ_d to 1, thereby removing the domain adaptation module from the domain adaptive anomaly scoring in DP-DGAD. (3) **w/o CDGA** removes the confident detection guided memory buffer update on the target dataset, using the model and buffer pretrained on source datasets and tested directly on target datasets. (4) **w/o Wiki** pretrains DP-DGAD on MOOC only. (5) **w/o MOOC** pretrains DP-DGAD on Wiki only.

From the visualization of the ablation study in Table 3, we have the following observations. **(1) DP-DGAD captures domain-specific patterns, resulting in better performance.** Compared to the ablation variants w/o DPA, w/o DPAD, and w/o CDGA, the full model demonstrates superior generalizability. This is because dynamic prototype-based anomaly scoring continuously evolves as the prototype buffer updates. As a result, it adapts better to new domains by capturing changing patterns over time. In contrast, w/o DPA uses a standard binary classifier. This classifier

¹<https://github.com/Jackmeory/WWW2026-DP-DGAD>

Table 2: Overall model performance across eight datasets with three anomaly percentages.

Metrics	Methods	AS-Topology			Bitcoin-Alpha			Email-DNC			UCI Messages			Bitcoin-OTC			TAX51			DBLP			Synthetic-Hijack			
		1%	5%	10%	1%	5%	10%	1%	5%	10%	1%	5%	10%	1%	5%	10%	1%	5%	10%	1%	5%	10%	1%	5%	10%	
AUROC	DGADs	SAD	40.45	42.27	46.43	59.03	64.12	63.77	52.66	54.02	58.82	59.63	58.80	62.44	72.21	74.59	77.13	44.50	46.12	52.26	45.70	44.24	40.97	62.51	64.09	69.52
		SLADE	41.09	45.56	48.28	66.78	65.05	64.93	49.34	51.67	55.51	55.72	57.50	59.82	70.17	69.20	74.77	47.59	48.64	50.96	45.01	43.22	41.67	66.67	61.70	63.02
		FALCON	48.05	53.43	55.16	58.77	64.80	67.55	80.33	86.21	85.74	66.95	67.65	71.78	66.17	70.86	73.07	51.23	54.57	53.20	48.56	53.77	51.21	59.19	63.99	68.48
		GeneralDyG	43.45	47.73	49.93	66.60	69.83	70.23	52.76	54.88	57.21	51.09	54.85	59.74	73.95	72.14	75.54	53.15	50.41	50.39	51.73	47.48	48.87	58.15	51.09	52.94
		TADDY	41.32	44.68	42.16	42.27	46.53	44.42	51.20	53.74	57.18	50.25	53.97	56.85	47.82	49.02	49.77	41.37	43.66	47.39	40.97	41.55	39.69	60.76	61.20	63.27
	Generalists	GraphPrompt	49.60	51.21	53.47	46.12	46.75	45.33	69.30	71.04	74.45	60.74	59.28	59.26	50.66	52.79	54.33	53.48	55.72	58.61	60.64	61.07	62.33	72.16	76.28	74.19
		UNPrompt	52.19	53.62	57.03	45.87	47.92	48.66	70.22	74.08	73.53	58.60	59.14	60.04	47.54	52.13	53.09	54.63	55.57	57.78	62.51	64.82	64.53	72.30	75.66	76.20
		AnomalyGFM	53.97	54.76	58.23	47.84	47.72	50.21	72.90	73.44	75.62	60.76	61.82	60.87	54.30	52.72	52.45	55.94	56.46	56.24	61.51	60.66	60.49	74.34	79.37	77.31
		ARC	54.33	55.86	60.28	41.14	43.64	45.19	71.32	77.83	73.27	58.79	60.04	59.43	48.51	53.85	50.69	48.49	54.07	58.71	62.98	67.57	63.15	71.74	74.89	75.77
		DP-DGAD	71.27	73.89	75.26	76.56	80.23	81.09	87.46	90.03	88.87	70.19	72.24	75.71	85.54	91.04	90.28	57.24	65.19	63.21	68.66	68.32	67.50	90.09	90.25	91.02
AUPRC	DGADs	SAD	6.16	13.24	11.77	6.99	9.01	10.99	13.32	19.33	29.69	12.22	20.67	25.25	7.32	12.97	18.85	5.27	9.18	10.06	3.64	5.54	9.80	12.45	17.01	26.79
		SLADE	6.10	12.27	10.54	5.05	9.78	11.41	14.76	21.22	27.95	12.77	19.87	24.02	5.11	9.32	11.04	4.95	6.71	12.29	2.12	6.90	10.14	10.10	15.21	20.67
		FALCON	7.66	15.29	12.11	5.12	7.90	14.97	16.24	23.56	33.10	12.33	24.59	26.15	4.52	11.72	13.16	11.59	13.48	14.11	5.37	11.16	9.68	12.72	15.77	23.32
		GeneralDyG	6.33	20.27	22.90	5.07	5.03	10.23	12.67	15.52	30.16	14.31	21.43	22.14	8.22	14.69	19.12	5.02	4.64	9.38	3.11	7.41	8.95	13.28	18.42	28.64
		TADDY	5.99	11.50	13.22	4.41	6.95	11.13	13.06	17.33	28.90	11.55	18.49	23.27	5.14	10.78	12.65	5.41	8.95	10.13	2.89	6.42	8.17	11.09	10.54	23.48
	Generalists	GraphPrompt	11.56	13.05	20.96	8.48	10.79	12.07	20.83	26.97	37.74	20.05	24.57	27.43	4.32	10.02	16.89	10.53	12.35	14.86	6.27	8.57	9.15	10.97	14.43	27.62
		UNPrompt	12.06	14.63	21.21	9.54	10.70	12.52	21.06	25.18	38.01	21.40	26.01	30.35	5.15	12.70	18.49	11.21	12.66	14.39	7.92	8.55	10.90	13.28	16.08	28.73
		AnomalyGFM	13.3	15.01	22.46	7.94	13.15	12.26	22.84	28.38	40.72	21.05	25.59	31.51	4.32	12.60	19.50	11.20	14.32	13.99	6.27	10.98	11.71	11.04	11.93	31.27
		ARC	12.04	19.98	26.65	12.07	11.06	13.35	16.48	28.35	41.71	19.47	24.87	29.71	6.89	14.99	20.45	8.98	8.66	15.42	8.45	9.22	10.44	15.53	16.79	25.02
		DP-DGAD	19.14	23.34	28.67	17.40	17.69	18.07	34.18	41.59	45.17	37.56	43.22	45.03	17.45	21.85	24.52	13.06	17.13	22.23	11.17	13.02	13.56	20.17	27.24	35.11

Table 3: Ablation results of different variants of DP-DGAD.

Metric	Variants	Dataset							
		AS-Topology	Bitcoin-Alpha	Email-DNC	UCI Messages	Bitcoin-OTC	TAX51	DBLP	Synthetic-Hijack
AUROC	w/o DPA	48.13	65.54	60.74	58.01	73.27	43.21	48.07	53.68
	w/o DPAD	36.40	38.72	43.95	32.40	49.24	27.80	31.59	44.64
	w/o CDGA	60.01	73.24	78.31	65.17	78.34	57.42	47.51	71.74
	w/o Wiki	66.60	46.36	63.39	54.23	48.55	45.86	49.90	65.82
	w/o MOOC	58.18	64.88	49.01	63.09	61.08	55.97	33.21	57.43
	DP-DGAD	75.26	81.09	88.87	75.71	90.28	63.21	67.50	91.02
AUPRC	w/o DPA	20.65	13.67	30.99	20.12	17.91	9.32	9.49	30.24
	w/o DPAD	7.01	9.67	11.47	10.13	8.44	5.90	12.78	13.68
	w/o CDGA	21.26	11.10	32.53	27.86	17.31	15.67	7.99	26.71
	w/o Wiki	21.46	12.29	28.22	23.24	15.57	11.44	11.11	27.03
	w/o MOOC	19.62	12.08	19.73	29.06	14.97	13.15	9.07	20.77
	DP-DGAD	28.67	18.07	45.17	45.03	24.52	22.23	13.56	35.11

Table 4: Different confidence generation strategies' performance.

Metric	Strategies	Dataset							
		AS-Topology	Bitcoin-Alpha	Email-DNC	UCI Messages	Bitcoin-OTC	TAX51	DBLP	Synthetic-Hijack
AUROC	Random Selection	52.73	58.21	64.13	53.48	48.06	45.18	48.92	65.33
	Threshold Based	61.42	65.89	71.08	60.95	65.78	52.76	55.84	73.81
	Distance Based	41.95	49.67	51.01	45.23	50.04	38.92	41.36	52.45
	Similarity Based	38.17	43.52	42.60	41.66	52.12	35.21	37.58	49.72
	DP-DGAD	75.26	81.09	88.87	75.71	90.28	63.21	67.50	91.02
AUPRC	Random Selection	18.94	11.85	20.38	25.67	9.97	13.76	8.42	22.54
	Threshold Based	22.13	13.24	14.48	31.85	8.77	16.89	10.38	27.93
	Distance Based	15.67	8.96	8.54	19.22	9.09	10.45	6.73	18.76
	Similarity Based	13.28	7.43	5.43	16.94	8.10	8.92	5.21	15.48
	DP-DGAD	28.67	18.07	45.17	45.03	24.52	22.23	13.56	35.11

only captures domain-agnostic patterns and cannot evolve to learn domain-specific patterns. As a result, it performs poorly on most datasets compared to DP-DGAD. Similarly, w/o DPAD cannot adapt

efficiently to new domains since it lacks the λ_e parameter that helps incorporate new domain patterns. Meanwhile, w/o CDGA fails to capture target domain patterns as it must rely solely on

domain-agnostic patterns. Consequently, both variants perform poorly compared to DP-DGAD. (2) **Multiple source datasets help capture domain agnostic patterns for better generalizability.** DP-DGAD is pretrained on both MOOC and Wiki datasets, unlike w/o Wiki and w/o MOOC, which use single-source datasets. By learning from multiple sources, DP-DGAD can retain shared prototypes that capture common anomalous patterns. These shared prototypes exhibit high similarity to anomalies across different domains, enabling effective generalization to new datasets. Consequently, DP-DGAD generalizes better to target datasets and achieves superior performance compared to single-source variants. More ablation studies can be found in Appendix C.

5.4 Pseudo Labeling Strategy Analysis

Comparison of Different Confident Detection Generation Methods. Apart from the important module, we also compare alternative strategies for selecting confident detection for pseudo-labels: (1) **Random Selection** randomly selects N_{con} pairs from the detection results. (2) **Threshold Based** uses fixed probability thresholds, selecting pairs with normal detection > 0.7 and normal detection < 0.3 . (3) **Distance Based** selects pairs with maximum Euclidean distance. (4) **Similarity Based** selects the pairs having the lowest Euclidean distance to the target datasets.

From the visualization of different strategies' performance in Table 4, we have the following observation. (1) **DP-DGAD generates more reliable pseudo labels, resulting in better performance.** DP-DGAD achieves 15-40% higher AUROC by selecting low-entropy (high-confidence) detections for pseudo-labeling. In contrast, distance-based and similarity-based methods select pairs based on geometric properties. However, they neglect detection reliability, introducing noise that degrades performance. These geometric methods perform 35-55% worse than DP-DGAD, often underperforming even random selection. This demonstrates that spatial relationships in latent space are unreliable indicators for pseudo-label quality. Threshold-based selection, using fixed probability cutoffs, achieves 75-85% of DP-DGAD's effectiveness. While less sophisticated, it captures detection confidence. However, fixed thresholds cannot adapt to varying confidence distributions, unlike DP-DGAD's dynamic entropy-based approach.

5.5 Hyperparameter Analysis

To analyze DP-DGAD's sensitivity to hyperparameters, we examine three key parameters: the dynamic prototype memory buffer size M , the momentum α , and the top confident detection number N_{con} used for pseudo labeling (Here we set it as a different proportion to the full dataset size). We vary these values and observe their impact on model performance in Fig. 4. Our findings are as follows. (1) **Increasing memory buffer size will improve performance.** This is due to the reason that a larger memory buffer will include more prototype pairs, thus covering broader aspects of the distribution of animal patterns. However, we observe an intriguing pattern on some datasets, that Larger buffer sizes actually degrade performance. This occurs because oversized buffers may include irrelevant prototype pairs, which are neither similar nor distinct enough from new domains. They introduce bias and interfere with

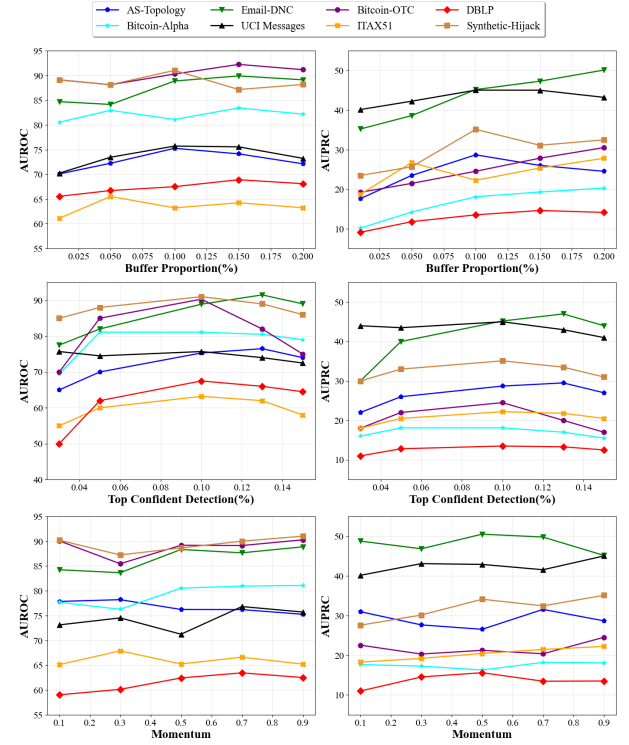


Figure 4: Impact of dynamic prototype memory buffer size M , confident detection N_{con} and momentum α on the DP-DGAD performance.

anomaly scoring, which relies on buffer prototypes to extract distributions. (2) **Using the top 10% most confident detections appears to be the optimal setting.** On most datasets, performance improves as the top confident detection number increases. However, performance begins to drop when this percentage exceeds 10%. This decline occurs because including more than the top 10% of detections reduces reliability. These less confident detections have high entropy and introduce additional noise into the pseudo-labeling process. (3) **DP-DGAD remains stable under different momentum.** We vary the momentum value and find DP-DGAD remains stable across all datasets with no clear trend as momentum increases. This stability arises from the fact that the memory buffer already captures sufficient similar abnormal and normal patterns through the selection score s_r constraint. As a result, the extracted distributions remain stable regardless of α 's value.

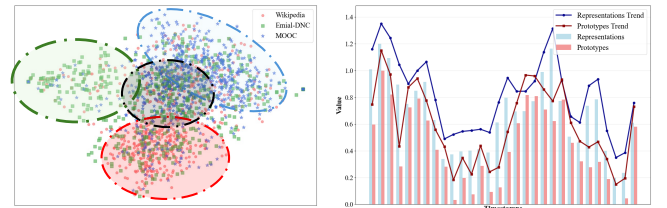


Figure 5: Visualization of prototypes in memory buffer and how they evolve over time.

5.6 Visualization

We visualize dynamic prototypes from DP-DGAD collected during the pre-training on MOOC, Wikipedia, and update Email-DNC. Besides, we also show how dynamic prototypes and edge representations evolve during the training (Fig. 5).

Our key observations are: (1) Dynamic prototypes capture both domain-specific patterns (three distinct regions dominated by each dataset's prototypes) and domain-agnostic patterns (central region with mixed prototypes from all datasets). (2) Prototypes evolve similarly to representations, demonstrating their temporal adaptability in capturing anomaly patterns.

6 Conclusion

In this paper, we propose a generalist DGAD detector that is trained on labeled source datasets and then generalized to target datasets from new domains without labels. Specifically, we aim to capture the evolving anomalous patterns that are both domain agnostic and domain specific, thereby achieving better generalizability compared to existing methods. To this end, we introduce DP-DGAD, a generalist detector that leverages dynamic prototypes extracted from temporal ego-graphs to model evolving anomalies. We further update memory buffer using confident pseudo-labels in a parameter free manner, enabling effective adaptation to unlabeled target domain. Extensive experiments across 10 real-world datasets from diverse domains validate the effectiveness of DP-DGAD in addressing the generalist DGAD problem.

Acknowledgments

This work is supported in part by Hong Kong RGC Theme-based Research Scheme (No.: T41-603/20-R); and in part by the Research Institute for Artificial Intelligence of Things, The Hong Kong Polytechnic University .

References

- [1] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural temporal graph neural networks for anomaly detection in dynamic graphs. In *Proceedings of the 30th ACM international conference on Information & Knowledge Management*. 3747–3756.
- [2] João Carvalho, Mengtao Zhang, Robin Geyer, Carlos Cotrini, and Joachim M Buhmann. 2023. Invariant anomaly detection under distribution shifts: a causal perspective. *Advances in Neural Information Processing Systems* 36 (2023), 56310–56337.
- [3] Dong Chen, Xiang Zhao, and Weidong Xiao. 2024. Fine-grained anomaly detection on dynamic graphs via attention alignment. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3178–3190.
- [4] Yankai Chen, Quoc-Tuan Truong, Xin Shen, Jin Li, and Irwin King. 2024. Shopping trajectory representation learning with pre-training for e-commerce customer understanding and recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 385–396.
- [5] Yankai Chen, Taotao Wang, Yixiang Fang, and Yunyu Xiao. 2025. Semi-supervised node importance estimation with informative distribution modeling for uncertainty regularization. In *Proceedings of the ACM on Web Conference 2025*. 3108–3118.
- [6] Yankai Chen, Menglin Yang, Yingxue Zhang, Mengchen Zhao, Ziqiao Meng, Jianye Hao, and Irwin King. 2022. Modeling scale-free graphs with hyperbolic geometry for knowledge-aware recommendation. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 94–102.
- [7] Zining Chen, Xingshuang Luo, Weiqiu Wang, Zhicheng Zhao, Fei Su, and Aidong Men. 2025. Filter or compensate: Towards invariant representation from distribution shift for anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 2420–2428.
- [8] Leyan Deng, Defu Lian, Zhenya Huang, and Enhong Chen. 2022. Graph convolutional adversarial networks for spatiotemporal anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems* 33, 6 (2022), 2416–2428.
- [9] Xingzhi Guo, Baojian Zhou, and Steven Skiena. 2022. Subset node anomaly tracking over large dynamic graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 475–485.
- [10] Zhongxuan Han, Xiaolin Zheng, Chaochao Chen, Wenjie Cheng, and Yang Yao. 2023. Intra and inter domain hypergraph convolutional network for cross-domain recommendation. In *Proceedings of the ACM Web Conference 2023*. 449–459.
- [11] Bowei He, Xu He, Yingxue Zhang, Ruiming Tang, and Chen Ma. 2023. Dynamically expandable graph convolution for streaming recommendation. In *Proceedings of the ACM Web Conference 2023*. 1457–1467.
- [12] Lingbai Kong, Wengen Li, Hanchen Yang, Yichao Zhang, Jihong Guan, and Shuigeng Zhou. 2024. Causalfomer: An interpretable transformer for temporal causal discovery. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [13] Srikanth Kumar, Francesca Spezzano, VS Subrahmanian, and Christos Faloutsos. 2016. Edge weight prediction in weighted signed networks. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 221–230.
- [14] Srikanth Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.
- [15] Jongha Lee, Sunwoo Kim, and Kijung Shin. 2024. Slade: Detecting dynamic anomalies in edge streams without labels via self-supervised learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1506–1517.
- [16] Aodong Li, Chen Qiu, Marius Kloft, Padhraic Smyth, Maja Rudolph, and Stephan Mandt. 2023. Zero-shot anomaly detection via batch normalization. *Advances in Neural Information Processing Systems* 36 (2023), 40963–40993.
- [17] Dongyuan Li, Satoshi Kosugi, Ying Zhang, Manabu Okumura, Feng Xia, and Renhe Jiang. 2025. Revisiting dynamic graph clustering via matrix factorization. In *Proceedings of the ACM on Web Conference 2025*. 1342–1352.
- [18] Dongyuan Li, Shiyin Tan, Ying Zhang, Ming Jin, Shirui Pan, Manabu Okumura, and Renhe Jiang. 2025. DyG-Mamba: Continuous State Space Modeling on Dynamic Graphs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=ja2wA4UncJ>
- [19] Yixin Liu, Shiyuan Li, Yu Zheng, Qingfeng Chen, Chengqi Zhang, and Shirui Pan. 2024. ARC: a generalist graph anomaly detector with in-context learning. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. 50772–50804.
- [20] Yixin Liu, Shirui Pan, Yu Guang Wang, Fei Xiong, Liang Wang, Qingfeng Chen, and Vincent CS Lee. 2021. Anomaly detection in dynamic graphs via transformer. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2021), 12081–12094.
- [21] Zhe Liu, Xiang Huang, Jingyun Zhang, Zhifeng Hao, Li Sun, and Hao Peng. 2024. Multivariate time-series anomaly detection based on enhancing graph attention networks with topological analysis. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1555–1564.
- [22] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM web conference 2023*. 417–428.
- [23] Xiaoxiao Ma, Ruikun Li, Fanzhen Liu, Kaize Ding, Jian Yang, and Jia Wu. 2024. Graph anomaly detection with few labels: A data-centric approach. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2153–2164.
- [24] Chaoxi Niu, Hezhe Qiao, Changlu Chen, Ling Chen, and Guansong Pang. 2024. Zero-shot generalist graph anomaly detection with unified neighborhood prompts. *arXiv preprint arXiv:2410.14886* (2024).
- [25] Tore Opsahl and Pietro Panzarasa. 2009. Clustering in weighted networks. *Social networks* 31, 2 (2009), 155–163.
- [26] Junjun Pan, Yixin Liu, Xin Zheng, Yizhen Zheng, Alan Wee-Chung Liew, Fuyi Li, and Shirui Pan. 2025. A label-free heterophily-guided approach for unsupervised graph fraud detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12443–12451.
- [27] Hezhe Qiao, Chaoxi Niu, Ling Chen, and Guansong Pang. 2025. AnomalyGFM: Graph foundation model for zero/few-shot anomaly detection. *arXiv preprint arXiv:2502.09254* (2025).
- [28] Ziyue Qiao, Qianyi Cai, Hao Dong, Jiawei Gu, Pengyang Wang, Meng Xiao, Xiao Luo, and Hui Xiong. 2025. GCAL: Adapting Graph Models to Evolving Domain Shifts. In *Forty-second International Conference on Machine Learning*.
- [29] Ryan Rossi and Nesreen Ahmed. 2015. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 29.
- [30] Haopeng Sun, Lumin Xu, Sheng Jin, Ping Luo, Chen Qian, and Wentao Liu. 2024. Program: Prototype graph model based pseudo-label learning for test-time adaptation. In *The twelfth international conference on learning representations*.
- [31] Sheng Tian, Jihai Dong, Jintang Li, Wenlong Zhao, Xiaolong Xu, Bowen Song, Changhua Meng, Tianyi Zhang, Liang Chen, et al. 2023. Sad: Semi-supervised anomaly detection on dynamic graphs. *arXiv preprint arXiv:2305.13573* (2023).
- [32] Yuntong Tian, Jiaxi Li, Huazhu Fu, Lei Zhu, Lequan Yu, and Liang Wan. 2024. Self-mining the confident prototypes for source-free unsupervised domain adaptation in image segmentation. *IEEE Transactions on Multimedia* 26 (2024), 7709–7720.

- [33] Yuxing Tian, Yiyang Qi, and Fan Guo. 2024. Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction. In *The twelfth international conference on learning representations*.
- [34] Guancheng Wan, Wenke Huang, and Mang Ye. 2024. Federated graph learning under domain shift with generalizable prototypes. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 15429–15437.
- [35] Huan Wang, Junyang Chen, Yirui Wu, Victor CM Leung, and Di Wang. 2025. Epm: Evolutionary perception method for anomaly detection in noisy dynamic graphs. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [36] Hongjun Wang and Yisen Wang. 2023. Generalist: Decoupling natural and robust generalization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 20554–20563.
- [37] Yuwen Wang, Shunyu Liu, Tongya Zheng, Kaixuan Chen, and Mingli Song. 2024. Unveiling global interactive patterns across graphs: Towards interpretable graph neural networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3277–3288.
- [38] Yuxia Wu, Lizi Liao, and Yuan Fang. 2025. Retrieval augmented generation for dynamic graph modeling. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1434–1443.
- [39] Feng Xia, Ciyuan Peng, Jing Ren, Fahih Gozi Febrinanto, Renqiang Luo, Vidya Saikrishna, Shuo Yu, and Xiangjie Kong. 2025. Graph Learning. *Foundations and Trends® in Signal Processing* 19, 4 (2025), 371–551.
- [40] Jiachong Xu, Shao-Yuan Lo, Bardia Safaei, Vishal M Patel, and Isht Dwivedi. 2025. Towards zero-shot anomaly detection and reasoning with multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 20370–20382.
- [41] Yiming Xu, Bin Shi, Teng Ma, Bo Dong, Haoyi Zhou, and Qinghua Zheng. 2023. Cldg: Contrastive learning on dynamic graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 696–707.
- [42] Hanchen Yang, Jiannong Cao, Wengen Li, Shuyu Wang, Hui Li, Jihong Guan, and Shuigeng Zhou. 2025. Spatial-Temporal Data Mining for Ocean Science: Data, Methodologies and Opportunities. *ACM Transactions on Knowledge Discovery from Data* (July 2025), 1–46.
- [43] Hanchen Yang, Yuwei Chen, Jiannong Cao, Wengen Li, Yu Yang, Shuyu Wang, Jihong Guan, Rufu Qin, and Shuigeng Zhou. 2024. UniOcean: A Unified Framework for Predicting Multiple Ocean Factors of Varying Temporal Scales. *IEEE Transactions on Geoscience and Remote Sensing* (2024).
- [44] Jaewon Yang and Jure Leskovec. 2012. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD workshop on mining data semantics*. 1–8.
- [45] Xiao Yang, Xuejiao Zhao, and Zhiqi Shen. 2025. A generalizable anomaly detection method in dynamic graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 22001–22009.
- [46] Shuo Yu, Huafei Huang, Yanming Shen, Pengfei Wang, Qiang Zhang, Ke Sun, and Honglong Chen. 2024. Formulating and representing multiagent systems with hypergraphs. *IEEE Transactions on Neural Networks and Learning Systems* 36, 3 (2024), 4599–4613.
- [47] Shuo Yu, Yicong Li, Shuo Wang, Tao Tang, Qiang Zhang, Jingjing Zhou, Ivan Lee, and Feng Xia. 2025. CaGE: A causality-inspired graph neural network explainer for recommender systems. *ACM Transactions on Information Systems* 43, 6 (2025), 1–29.
- [48] Shuo Yu, Yingbo Wang, Ruolin Li, Guchun Liu, Yanming Shen, Shaoxiong Ji, Bowen Li, Fengling Han, Xiuzhen Zhang, and Feng Xia. 2025. Graph2text or graph2token: A perspective of large language models for graph learning. *ACM Transactions on Information Systems* (2025).
- [49] Shuo Yu, Yingbo Wang, Zhitao Wan, Yanming Shen, Qiang Zhang, and Feng Xia. 2025. Path-aware Few-shot Knowledge Graph Completion. *IEEE Transactions on Artificial Intelligence* (2025).
- [50] Beichuan Zhang, Raymond Liu, Daniel Massey, and Lixia Zhang. 2005. Collecting the Internet AS-level topology. *ACM SIGCOMM Computer Communication Review* 35, 1 (2005), 53–61.
- [51] Xinni Zhang, Yankai Chen, Cuiyun Gao, Qing Liao, Shenglin Zhao, and Irwin King. 2022. Knowledge-aware neural networks with personalized feature referencing for cold-start recommendation. *arXiv preprint arXiv:2209.13973* (2022).
- [52] Xiaokun Zhang, Bo Xu, Zhaochun Ren, Xiaochen Wang, Hongfei Lin, and Fenglong Ma. 2024. Disentangling id and modality effects for session-based recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 1883–1892.
- [53] Xiaokun Zhang, Bo Xu, Youlin Wu, Yuan Zhong, Hongfei Lin, and Fenglong Ma. 2024. Finerec: Exploring fine-grained sequential recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 1599–1608.
- [54] Haihong Zhao, Aochuan Chen, Xiangguo Sun, Hong Cheng, and Jia Li. 2024. All in one and one for all: A simple yet effective method towards cross-domain graph pretraining. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4443–4454.
- [55] Jialun Zheng, Divya Saxena, and Jiannong Cao. 2025. COIN-GNN: Inductive Spatial-Temporal Prediction for Continuous Distribution Shifts via Graph Neural Networks. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [56] Jialun Zheng, Divya Saxena, Jiannong Cao, Hanchen Yang, and Penghui Ruan. 2024. Inductive Spatial Temporal Prediction Under Data Drift with Informative Graph Neural Network. In *International Conference on Database Systems for Advanced Applications*. Springer, 169–185.
- [57] Jiawen Zhu and Guansong Pang. 2024. Toward generalist anomaly detection via in-context residual learning with few-shot sample prompts. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 17826–17836.
- [58] Zulun Zhu, Kai Wang, Haoyu Liu, Jintang Li, and Siqiang Luo. 2024. Topology-monitorable contrastive learning on dynamic graphs. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4700–4711.

A Detailed Experiments Description

A.1 Datasets

For a dataset with no ground truth anomalies. We follow previous work [20] to inject anomalies in random timestamps of the dataset. Specifically, take p to be the proportion of the total m number of samples in the dataset. We link $p \times m$ number of original disconnected nodes to be the injected anomalies.

Wikipedia [14]: The Wikipedia dataset consists of one month of edits made on Wikipedia pages. It selects 1,000 most edited pages and editors who made at least 5 edits (a total of 8,227 users), generating 157,474 interactions. Anomalies within this dataset are identified based on public ground-truth labels of banned users, resulting in 217 positive (anomalous) labels.

Bitcoin-OTC and Bitcoin-Alpha [13]: The Bitcoin-Alpha and Bitcoin-OTC datasets, collected from two Bitcoin platforms (Alpha and OTC), represent users as nodes and trust ratings between them as edges. These ratings, originally on a scale from -10 (fraudster) to +10 (high trust), are scaled to -1 to 1.

Email-DNC [29]: This is a network of emails from the 2016 Democratic National Committee leak, where nodes represent individuals and edges denote email relationships between each individual.

UCI Messages [50]: The UCI Messages dataset, collected from an online community platform, represents users as nodes and messages exchanged between them as edges.

AS-Topology [25]: AS-Topology is a network connection dataset collected from autonomous systems of the Internet. Nodes represent autonomous systems, and edges denote connections between these autonomous systems.

MOOC [14]: The MOOC dataset captures student online course-related actions, such as viewing videos or submitting answers. It comprises 7,047 users interacting with 98 course items, totaling over 411,749 interactions, and includes 4,066 recorded drop-out events recorded as anomalies.

Synthetic-Hijack [15]: The Synthetic-Hijack dataset is a modified version of the Email-EU dataset, which captures email exchanges within a large European research institution. In the Synthetic-Hijack dataset, certain normal user accounts are "hijacked" at a specific point in time. These accounts then begin to exhibit anomalous behavior, such as sending spam emails to various recipients.

TAX51 [41]: Derived from tax transaction networks, it represents companies as nodes and their transaction relationships as edges.

DBLP [44]: The DBLP collaboration network is a dataset where authors are represented as nodes, and an edge exists between authors who have co-authored a paper. Publication venues (conferences) serve as ground-truth communities, acting as proxies for highly overlapping scientific communities within the network.

A.2 Baselines

GeneralDyG [45]: For fair comparison, we modify GeneralDyG, which was originally under an unsupervised learning setting, by adding an additional transformer module to compare with abnormal neighbors in labeled source datasets. Then it was updated using only normal samples on target datasets before testing.

FALCON [3]: Originally unsupervised, assuming only normal training data, FALCON was modified by integrating label information into its contrastive learning module alongside temporal patterns for better distinguishability in source datasets. Finally, it was updated using only normal samples on target datasets before testing.

SLADE [15]: On source datasets, SLADE integrates a classification BCE loss with its self-supervised learning to enhance the discriminability. Then, it adapts its learned normal patterns solely through self-supervision using normal samples on target datasets.

SAD [31]: SAD integrates its deviation loss with label-enabled contrastive learning to enhance discriminability. While on target datasets, it adapts solely by updating the memory bank and applying pseudo-label contrastive learning to refine normal patterns.

TADDY [20]: Here we follow the essence of TADDY to represent a dynamic graph using GNN and Transformer for embedding, then guided it with BCE loss on source datasets. It is updated on the target dataset using a similar confident detection-based pseudo-labeling strategy as DP-DGAD.

ARC [19]: To adapt ARC for dynamic graphs, we add an additional transformer temporal-aware encoder. For labeled source datasets, the cross-attention module would leverage both normal and abnormal labels to learn robust anomaly scoring. On unlabeled target datasets, assuming all samples are normal, ARC would continuously refine its internal representation of normal dynamic patterns via the cross-attention module.

AnomalyGFM [27]: We add an additional transformer temporal-aware encoder. For pre-training on labeled source dynamic datasets, it would learn graph-agnostic normal and abnormal prototypes from these temporal residuals, utilizing both its alignment and classification losses. For unlabeled target dynamic datasets, assuming all samples are normal, the pre-trained encoder and abnormal prototype would be frozen, while the normal prototype would be refined through few-shot prompt tuning using the target dataset's "normal" temporal residuals, enabling robust anomaly scoring.

UNPrompt [24]: For training on labeled source datasets, the backbone model consists of GNN and transformer, which would be pre-trained via dynamic graph contrastive learning, and neighborhood prompts would learn generalized patterns based on the predictability of a node's attributes from its temporal neighbors.

GraphPrompt [22]: We replace the backbone with a GNN and a transformer for temporal subgraphs. For labeled source data, a learnable prompt guides the backbone model to align temporal subgraph representations with normal and abnormal prototypes for anomaly scoring. For unlabeled targets (assumed normal), the prompt would be refined via normal samples to adapt the normal prototype to the new dynamic environment for continuous detection.

B Algorithms

Algorithms for pretraining on source datasets and updating on target datasets are presented in Algorithms 1 and 2.

Algorithm 1 Pretrain DP-DGAD on source datasets

Input: Source dynamic graph datasets $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_S\}$; Pre-training epoch $\mathcal{I}$.

Output: Function $\Psi_{\text{pretrain}}; \mathcal{B}$.

```

1: Initialize memory buffer  $\mathcal{B} \leftarrow \emptyset$ .
2: Initialize GNN and Transformer backbone model.
3: for datasets  $\mathcal{G}_s \in \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_S\}$  do
4:   for  $i = 1, \dots, \mathcal{I}$  do
5:     if  $\mathcal{B}$  is not empty then
6:        $p_n, p_a, \mu_{n,t}, \mu_{a,t}, \Sigma_{n,t}, \Sigma_{a,t} \leftarrow$  Initialization from  $\mathcal{B}$ 
7:     else
8:        $p_n, p_a, \mu_{n,t}, \mu_{a,t}, \Sigma_{n,t}, \Sigma_{a,t} \leftarrow$  random Initialization
9:     end if
10:    for  $t \in \{1, 2, \dots, T\}$  do
11:      for  $e_i \in E_t$  do
12:        Extract  $k$ -hop ego-graph  $\mathcal{G}_{ego}$  around  $e_i$ 
13:        Obtain  $h_i$  using Eq.2, normalize it using Eq.3
14:        Get representation  $z_i$  of  $h_i$  via Eq.4
15:        Compute scores  $s_i$  using Eq.15-16, then get  $L_{BCE}$ 
16:      end for
17:    end for
18:    Align  $p_n, p_a$  with  $Z_n, Z_a$  using  $L_A$  in Eq.5
19:    Update  $\mu_{n,t}, \mu_{a,t}, \Sigma_{n,t}, \Sigma_{a,t}$  using Eq.9-14
20:    Update model with  $L_{BCE}$  and Alignment loss  $L_A$ 
21:    if  $\mathcal{G}_s$  is not first source dataset then
22:      Compute  $s_e, s_d$  using Eq.6-7
23:      Compute ranking score  $s_r = \lambda_d \cdot s_d - \lambda_e \cdot s_e$ 
24:      if  $\mathcal{B}$  is full then
25:        if  $s_r >$  lowest  $s_r$  in  $\mathcal{B}$  then
26:          replace the lowest pair with new  $p_n, p_a$ 
27:        else
28:          Maintain  $\mathcal{B}$ 
29:        end if
30:      else
31:        Add  $p_n, p_a$  to  $\mathcal{B}$ 
32:      end if
33:    else
34:      Compute difference score  $s_d$  using Eq.6
35:      if  $\mathcal{B}$  is full then
36:        if  $s_d >$  lowest  $s_d$  in  $\mathcal{B}$  then
37:          replace the lowest pair with new  $p_n, p_a$ 
38:        else
39:          Maintain  $\mathcal{B}$ 
40:        end if
41:      else
42:        Add  $p_n, p_a$  to  $\mathcal{B}$ 
43:      end if
44:    end if
45:  end for
46: end for

```

C Additional Experiments Results

To further analyze DP-DGAD's sensitivity to hyperparameters, we examine two parameter pairs: the loss parameter λ_A and λ_{BCE} , the similarity and difference score parameter λ_d and λ_e . We vary

Table 5: Different proportions of the loss parameter’s impact on performance.

Metric	Proportion	Dataset							
		AS-Topology	Bitcoin-Alpha	Email-DNC	UCI Messages	Bitcoin-OTC	TAX51	DBLP	Synthetic-Hijack
AUROC	$\lambda_A = 1, \lambda_{BCE} = 0$	42.67	51.23	48.91	45.78	52.34	41.56	39.67	58.12
	$\lambda_A = 0.9, \lambda_{BCE} = 0.1$	57.82	64.56	62.13	69.45	64.91	55.34	51.78	73.29
	$\lambda_A = 0.7, \lambda_{BCE} = 0.3$	63.45	70.12	73.67	60.89	66.23	59.67	53.12	77.45
	$\lambda_A = 0.5, \lambda_{BCE} = 0.5$	67.13	66.89	64.91	63.23	76.78	60.45	65.89	78.12
	$\lambda_A = 0.3, \lambda_{BCE} = 0.7$	67.67	75.34	67.56	64.12	79.45	62.89	56.23	80.78
	$\lambda_A = 0.1, \lambda_{BCE} = 0.9$ (DP-DGAD)	75.26	81.09	88.87	75.71	90.28	63.21	67.50	91.02
	$\lambda_A = 0, \lambda_{BCE} = 1$	31.34	48.67	40.23	52.89	50.45	38.91	47.23	57.56
AUPRC	$\lambda_A = 1, \lambda_{BCE} = 0$	7.23	8.56	15.67	12.34	9.78	6.89	5.67	11.23
	$\lambda_A = 0.9, \lambda_{BCE} = 0.1$	13.89	12.34	29.23	20.12	18.67	10.45	9.78	19.67
	$\lambda_A = 0.7, \lambda_{BCE} = 0.3$	14.56	16.67	21.89	21.34	11.34	18.12	11.45	21.23
	$\lambda_A = 0.5, \lambda_{BCE} = 0.5$	10.78	16.23	32.34	23.67	22.89	12.78	11.89	32.45
	$\lambda_A = 0.3, \lambda_{BCE} = 0.7$	17.12	13.45	34.56	24.23	13.67	15.89	13.12	30.78
	$\lambda_A = 0.1, \lambda_{BCE} = 0.9$ (DP-DGAD)	28.67	18.07	45.17	45.03	24.52	22.23	13.56	35.11
	$\lambda_A = 0, \lambda_{BCE} = 1$	6.45	7.89	9.23	5.67	6.56	10.34	7.89	5.78

Table 6: Different proportion of λ_e and λ_d ’s impact on performance.

Metric	Proportion	Dataset							
		AS-Topology	Bitcoin-Alpha	Email-DNC	UCI Messages	Bitcoin-OTC	TAX51	DBLP	Synthetic-Hijack
AUROC	$\lambda_e = 0.3, \lambda_d = 0.7$	48.93	62.34	70.45	57.89	62.56	48.67	53.21	75.34
	$\lambda_e = 0.9, \lambda_d = 0.1$	51.45	69.87	61.23	51.34	64.12	40.45	51.89	73.67
	$\lambda_e = 0.5, \lambda_d = 0.5$	50.12	70.45	63.67	53.56	67.89	47.34	55.45	68.91
	$\lambda_e = 0.1, \lambda_d = 0.9$	42.67	68.23	75.12	59.23	66.34	51.89	60.78	67.23
	$\lambda_e = 0.7, \lambda_d = 0.3$ (DP-DGAD)	75.26	81.09	88.87	75.71	90.28	63.21	67.50	91.02
AUPRC	$\lambda_e = 0.3, \lambda_d = 0.7$	8.56	6.23	18.91	21.78	10.34	9.67	6.23	11.45
	$\lambda_e = 0.9, \lambda_d = 0.1$	9.12	7.89	21.34	19.45	8.78	11.12	5.67	19.78
	$\lambda_e = 0.5, \lambda_d = 0.5$	11.89	5.45	22.67	13.12	7.45	8.34	7.89	13.89
	$\lambda_e = 0.1, \lambda_d = 0.9$	12.34	9.78	20.23	12.67	9.89	10.78	7.45	12.67
	$\lambda_e = 0.7, \lambda_d = 0.3$ (DP-DGAD)	28.67	18.07	45.17	45.03	24.52	22.23	13.56	35.11

Algorithm 2 Update DP-DGAD on target datasets

Input: Target dynamic graph datasets $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K\}$; Training epoch $\mathcal{I}$; Function $\Psi_{pretrain}$; $\mathcal{B}$.

Output: Updated memory buffer $\mathcal{B}$.

```

1:
2: for  $t \in \{1, 2, \dots, T\}$  do
3:   for  $e_i \in E_t$  do
4:      $z_i \leftarrow$  Extract  $k$ -hop temporal ego-graph for embedding
5:     Obtain  $\mu_{n,t}, \mu_{a,t}, \Sigma_{n,t}, \Sigma_{a,t}$  from buffer  $\mathcal{B}$ 
6:     Compute  $s_{a,i}, s_{n,i}$  using Eq.16-17
7:     Compute anomaly scores  $s_i$ 
8:     Convert  $s_i$  to  $p_i$  and compute  $\mathcal{H}(p_i)$ 
9:   end for
10:  Select top  $N_{con}$  lowest entropy edges, whose detection result will serve as a pseudo label
11: end for
12: Initialize dynamic prototypes  $p_n, p_a$  from buffer  $\mathcal{B}$ 
13: for  $i = 1, \dots, \mathcal{I}$  do
14:   for each confident edge with pseudo-label do
15:     Get representation  $z_i$ 
16:   end for
17:   Update  $\mathcal{B}$  with  $s_r$  and  $p_n, p_a$  using  $L_A$ 
18: end for

```

these parameters and visualize them in Table 5 and Table 6. From these two tables, we have the following observations: (1) **Balanced alignment and classification losses are crucial for effective prototype learning.** The optimal configuration ($\lambda_A = 0.1$, $\lambda_{BCE} = 0.9$) significantly outperforms both extreme cases. Pure alignment ($\lambda_A = 1$) yields unacceptable AUROC (39.67-58.12%), lacking BCE’s discriminability guidance. Conversely, pure BCE ($\lambda_A = 0$) underperforms (47.23-67.56% AUROC) as prototypes fail to capture meaningful anomaly patterns without alignment. This highlights that excessive alignment hinders learning, while its complete absence prevents effective prototype-based scoring. The optimal balance ensures prototypes capture anomaly patterns and the embedding space is properly optimized. (2) **Cross-domain similarity outweighs within-domain distinctiveness for generalization.** DP-DGAD achieves optimal performance with $\lambda_e = 0.7$ and $\lambda_d = 0.3$, prioritizing similarity to new domains over prototype distinctiveness. This configuration enables the model to retain prototypes that generalize well by identifying shared anomalous patterns across domains. Conversely, overemphasizing the difference score (s_d) with higher λ_d values leads to selecting highly distinct but domain-specific prototype pairs that fail to transfer effectively, causing performance drops of 15-40% across datasets. This balance ensures DP-DGAD captures domain-agnostic patterns essential for cross-domain generalization while maintaining sufficient discriminative power.